

存储侧 CSA2 精确索引门控： Vortex-AXI/UCIe 链路的数值合同与瓶颈

匿名作者

数字硬件设计与可复现实验研究

摘要：将 CSA2 索引门控放在存储侧可能避免向计算侧搬运未选中的索引键；真实收益取决于精确评分代价、索引驻留状态和请求时延。本文建立 E2M1 FP4 分组缩放、有符号 BF16 加权 ReLU、稳定 Top-K 及错误完成的数值与事务合同，并以 Vortex 命令处理器 DMA、AXI256 / AXI2Flit / UCIe 和在线存储实现可检查的 RTL 路径。独立 H32/N640/K512 核心实验中，16 路设计的 444,525 周期有 389,120 周期用于算术 (87.5%)。在相同输入的 Vortex 在线系统中，原位位置排序将排序相关阶段从 60,872 降至 5,374 周期，完整命令从 670,720 降至 615,173 周期 (减少 8.28%)；两版均核对全部 512 条分数/索引记录与 147,456 B 聚集数据。保守候选上界只避免 0.13% 的头评分，未进入 RTL。结果给出评分、排序与搬运的瓶颈分解；相对计算侧实现的加速、能效与完整模型收益仍需公平对照。

关键词：数字集成电路；稀疏注意力；FP4；近存计算；RTL；AXI；UCIe

1 引言

长上下文索引键若驻留在远端存储，计算侧每次评分可能需要将大量最终未选中的键送过链路。把精确评分移至存储侧可以改变这部分数据流，却也会把算术、选择和事务完成时间放到远端关键路径。本文研究的问题是：在 CSA2 的真实数值格式和 Vortex 发起的 AXI / UCIe 访问路径下，能否完成可复现的精确门控，以及哪些成本阻碍它转化为端到端收益？输入记录必须实际送达 RTL，选中的数据必须完成读取，命令完成状态必须沿处理器的实际访存路径返回。

2026 年 9 月随 DeepSeek-V4.1-Flash 发布的 CSA2，提供了公开的模型配置、低精度数据格式和参考代码，因而适合作为本研究的近期算法对象^[1-2]。CSA2 在 DSA 所采用的可学习加权 ReLU 索引方法基础上，引入压缩表示以及跨层 KV 与索引复用^[3]。FULL、REINDEX、REUSE 和 Top-K 都属于已有算法语义；本文不将它们重新命名为新的算法。

本文以研究仓库中记录的固定版本 StorageStacked 为基础^[4]，收敛为 Vortex SimX 命令处理器经 AXI256、AXI2Flit 和 UCIe 访问在线存储的系统。gem5 系统调用仿真 (syscall emulation, SE) 中的宿主进程仅运行 Vortex 运行时与配置命令处理器；其程序与栈所在的本地页池不属于远端目标存储区域。门控寄存器与数据请求由 Vortex CP DMA 发出。该系统不是全芯片 RTL 模型，也没有验证 GPU 核函数直接发出门控命令。

本文的可验证贡献有三项。(1) 给出与官方格式相容、但明确独立于未指定 GPU 归约次序的 FP4 / FP32 数值合同，并在有符号权重、同分选择、次正规数和错误输入下对 RTL 逐位核对。(2) 把评分、稳定选择、按位置输出与 KV 聚集接入到同一个在线存储后端；DONE 等待真实写响应，Vortex CP DMA 的请求来源由轨迹验证。(3) 分离独立核心实验和在线系统实验，并以同配置在线消融量化位置排序优化，识别剩余评分与搬运瓶颈。本文尚未提出显著减少评分工作量的新机制，因此这些结果定位为后续算法与系统协同设计的可复现基线。

1.1 相关工作与研究边界

SpAtten 使用渐进量化、token / head 剪枝与专用 Top-K 引擎^[5]；PADE 使用不确定区间过滤与阶段融

合^[6]。StreamIndex 已为 CSA 索引器实现分块流式 Top-K，并在合成形状输入上报告内存占用^[7]；MISA 通过查询相关的索引头路由减少评分成本，但它改变了全头精确评分的执行集合^[8]。近期 GVR 利用解码步骤间相关性实现精确 Top-K，HPC-Ops Top-K 利用采样引导及完整行验证减少选择工作^[9-10]。在存储层次方面，HiSparse 以主机内存和有界 GPU 缓存管理被选中的 KV^[11]，IndexCache 利用跨层相似性复用选择结果^[12]。本文当前 RTL 对每个被接受的键计算全部 $H \times 128$ 个 FP4 乘积项，且保持定义好的位级合同。因此，“近存”位置、流式最小堆、低精度格式、选择复用或简单的头数削减都不能单独作为创新性证据，也不能从不同平台的论文结果直接推导本系统的速度优势。

2 目标算子与实现范围

对于单个查询，投影后的索引分数为

$$s_i = \sum_{h=0}^{H-1} w_h \text{ReLU} \left(\sum_{d=0}^{127} q_{h,d} k_{i,d} \right), \quad H \leq 32. \quad (1)$$

其中，所有索引头共享同一个索引键向量 k_i 。输入的 BF16 权重 w_h 已包含官方投影头权重及其 $D^{-1/2} H^{-1/2}$ 缩放因子，RTL 不再重复执行该归一化。学习得到的 w_h 可以为负，因此必须先完成每个头的点积，再应用 ReLU，随后进行有符号加权。将 ReLU 移至跨头求和之后会改变算法语义。本文不以各头贡献均非负这一错误假设进行提前剪枝。

官方 CSA2 配置采用 32 个索引头、128 维向量和 Top-512。本文的 N 表示压缩后的索引/KV 记录数， i 表示这些记录的原始位置，而非原始词元编号。当压缩索引与滑动窗口条目合并时，软件负责应用官方定义的偏移，并处理不可见条目对应的 -1 占位符。官方候选来源层构建每块含 8 个位置、最多 2048 个块的候选块池，并保留最新的不足一个整块的尾块。本文硬件接收展开后的有效候选索引列表，要求索引按升序排列且不重复。候选块池生成、压缩、Q/K 投影、均方根归一化 (RMSNorm)、旋转位置编码 (RoPE)、量化及稀疏注意力运算均由处理器或软件完成。在预填充阶段，软件先将块掩码与可见位置求交，再发起物理读取。下文所述硬件模式对应上述算子边界内的执行方式，并不表示已实现 CSA2 中

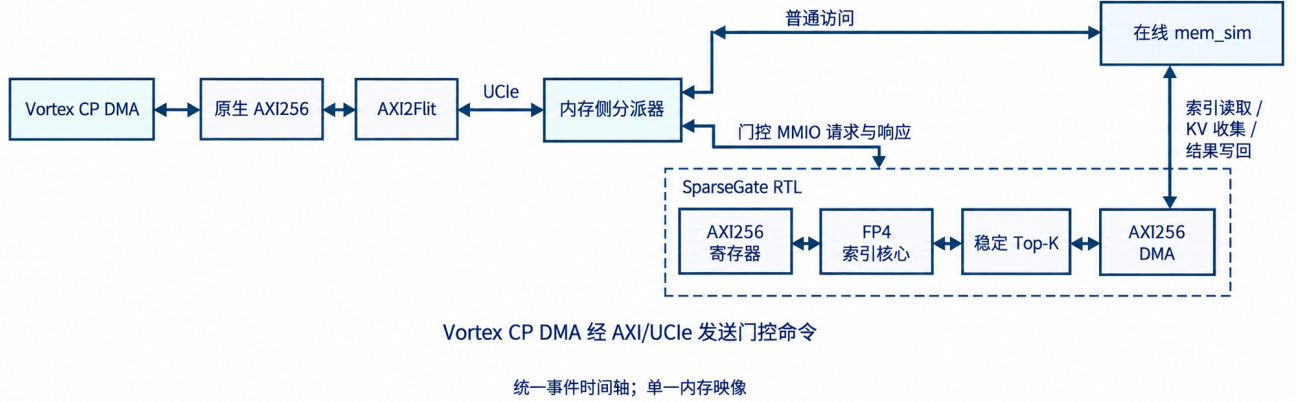


图 1: Vortex CP DMA 门控命令路径。门控 MMIO 经 AXI / UCle 到达 C++ 验证分发器与可综合的 SparseGate RTL。RTL DMA 与普通访问共用在线 mem_sim，响应沿同一链路返回。

全部层角色。

3 数值约定与微架构设计

3.1 紧凑记录格式与组内精确运算

索引器采用 E2M1 FP4 数值, 每 32 维共享一个 E8M0 缩放因子^[2,13]。FP4 并非有符号均匀 INT4 格式。将 E2M1 数值乘以 2 后, 可得整数集合

$$\mathcal{M} = \{0, \pm 1, \pm 2, \pm 3, \pm 4, \pm 6, \pm 8, \pm 12\}. \quad (2)$$

组内点积 $D_{h,g} = \sum_{j=0}^{31} m_{h,g,j}^q m_{i,g,j}^k$ 可在 $[-4608, 4608]$ 范围内精确计算。对于缩放字节 e_q, e_k , 该组的 FP32 贡献为

$$a_{h,g} = \text{RNE}_{32}(D_{h,g} 2^{e_q+e_k-256}). \quad (3)$$

其中, RNE_{32} 表示采用“舍入到最近值、中点取偶”的 FP32 舍入规则。实现中依次累加四个组的贡献, 应用 ReLU, 再与精确扩展为 FP32 的有限有符号 BF16 权重相乘, 最后按顺序累加各头结果。每次浮点运算均采用上述舍入规则, 并支持渐进下溢, 即保留可表示的次正规数。对于实际读取的记录, 若 E8M0 字节为 255、权重为非有限值, 或中间 FP32 结果溢出, 硬件将报错且不输出成功的选择结果。仅扫描候选位置时, 未读取的键记录不进行格式检查。

明确的数值约定使 RTL 与独立参考模型能够进行逐位比对, 但并不保证与采用其他累加顺序或 BF16 输出舍入的 GPU 内核逐位等价。分数相同时优先保留索引较小的条目, 正零与负零也按相等分数处理。

3.2 共享键缓冲与有界选择结构

查询 (Q) 缓冲存储 32 条记录, 每条包含 64 B 紧凑数值、4 B 缩放因子和 2 B 权重, 逻辑载荷共 2240 B。一个 68 B 的键 (K) 缓冲在所有索引头之间复用。物理存储中, 每条 Q 或 K 记录占 96 B, 即三个对齐的 AXI256 数据拍; 填充字节也计入传输量。每个参与评分的键记录只读取一次, 无需为每个索引头分别建立存储副本。

最小堆最多存储 512 对 32 位分数与 32 位索引, 逻辑载荷为 4 KiB。按上述稳定排序规则, 堆根始终为当前保留候选中最差的条目。新分数到达后, 执行追加、替

换堆根或丢弃操作; 堆调整需要 $O(\log K)$ 次比较, 每一步至多考察两个子节点。核心通过带有 valid/ready 握手和有限缓冲的接口传递分数与结果。封装模块将堆输出收集到独立的 4 KiB 结果缓存。原型基线通过逐项插入排序按索引升序输出, 最坏复杂度为 $O(K^2)$; 改进版在同一个结果缓存中按索引建立最大堆、逐个取出最大位置并原位排序, 复杂度为 $O(K \log K)$ 。分数始终随索引成对搬移, 排序不改变分数位模式或 Top-K 成员。原位堆排序是通用算法, 这里的研究对象是它对在线门控命令关键路径的实际影响, 而非排序算法的新颖性。

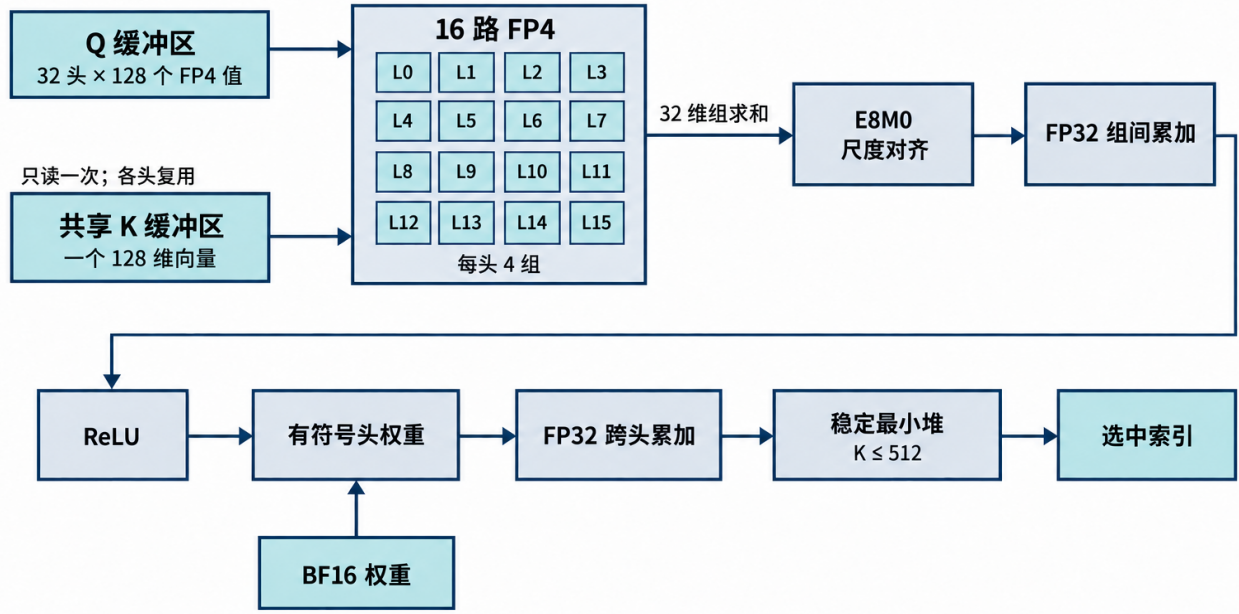
4 在线系统集成与命令完成规则

4.1 RTL 边界与协议适配

原存储目标通过 C++ FIFO 对象传递完整突发请求, 并不直接提供 RTL AXI 引脚。新增分发器将寄存器请求转换为实际的 AXI 从通道握手, 并将 RTL DMA 事务转换为对同一个 MemSimBackend 的请求。分发器不计算分数、不维护私有数据镜像, 也不预测存储访问何时完成。原生 mem_sim 接收请求后, 在后续返回数据字节与完成事件, 再由这些事件驱动 RTL 响应通道。

宿主软件负责配置 Vortex 命令处理器, 而输入上传、门控寄存器写入与状态读取、结果回读的总线请求均由该处理器的 DMA 主设备发出。设备地址 0x900f0000 经 Vortex BAR 映射至物理地址 0x1900f0000; 对应 RTL 模型采用匹配的高地址 1 MiB 数据窗口。

RTL 顶层采用 64 位地址、16 位事务标识、256 位数据和 32 位写字节选通信号 WSTRB。验收配置的存储地址窗口为 1 MiB, 其中预留 4 KiB 的存储器映射输入输出 (memory-mapped I/O, MMIO) 窗口; 各数据缓冲区必须完整落在配置的地址范围内。分发器拒绝独占访问请求。写地址 (AW) 与写数据 (W) 通道分别缓冲, 响应载荷在背压期间保持稳定。从接口支持自然对齐、宽度不超过 32 B 的单数据拍访问; 不支持的突发请求返回错误, 并完成必要的排空处理。DMA 每次仅发出一个对齐的 32 B INCR 数据拍。内部响应标签与处理器侧原有事务标识分离, 因此即使 DMA 使用 ID 1, 也不会产生标识混淆。



投影后的数值通路；数字标准单元实现

图 2: 实际算术数据通路。基准实现包含 16 路 FP4 运算单元，每组经过两批并行乘积运算后，再进行 FP32 缩放转换与累加。四个组构成一个头的点积；依次完成 ReLU、有符号加权和跨头归约后，分数才进入堆。存储阵列尚未绑定物理 SRAM 宏。

所有 Verilator 求值均发生在既有 SystemC 时钟边沿，与 gem5 共用飞秒级时间轴，且不链接第二套 SystemC 内核。普通流量保留独立队列，与 DMA 共同访问在线存储后端。当前实现最多保留一个未完成的 DMA 请求，这是明确的吞吐率限制。

4.2 执行模式与结果提交

FULL 模式扫描调用方给定的有效键区间。REINDEX 模式仅读取指定候选位置，并检查索引是否唯一、升序且未越界。REUSE 模式要求已经存在成功提交的选择结果，且上下文标识、版本号 (epoch)、键数量、头数量、K 值、Q 地址和 K 地址均匹配。该模式不读取 Q/K，也不重新评分，但仍执行结果写回和所请求的 KV 聚集读取与搬运。源数据发生变化时，软件必须递增版本号或使缓存失效。这一显式协议不提供硬件缓存一致性，也不是访问控制机制。

每个结果占一条 32 B 记录，包含索引、FP32 分数位模式及零填充。选中的紧凑主 KV 记录按索引顺序读取并复制到聚集输出区。只有所有输出写响应 (BRESP) 均成功后，硬件才提交 DONE 完成状态。失败命令可能已产生部分写入，但错误状态禁止调用方将这些数据作为完整结果使用。后续新命令可以从错误状态恢复；命令在途期间的外部复位恢复不在当前接口约定范围内。

4.3 传输量统计

设有效候选数量为 $1 \leq C \leq N$ ；REINDEX 不接受空列表。FULL 的结果数量为 $R = \min(K, N)$ ，REINDEX

为 $R = \min(K, C)$ ，REUSE 则沿用此前成功提交的结果数量。因此，在候选数量不足 K 的选择之后执行复用，仍保留此前较小的结果数量。若 KV 记录大小 V 为 32 B 的整数倍，物理 DMA 数据拍数量为

$$B_{\text{read}}^{\text{full}} = 3H + 3N + RV/32, \quad (4)$$

$$B_{\text{read}}^{\text{reindex}} = 3H + 3C + \lceil C/8 \rceil + RV/32, \quad (5)$$

$$B_{\text{read}}^{\text{reuse}} = RV/32, \quad (6)$$

$$B_{\text{write}} = R(1 + V/32). \quad (7)$$

上述计数不包含处理器侧的初始上传、MMIO、结果回读和 UCIE 成帧开销，这些流量在系统轨迹中另有记录。复用与分层候选均为 CSA2 已有机制，其传输量减少不构成本文新的算法发现。完整的架构比较还应纳入计算侧常驻索引缓存，而不能假设所有对比方案每次均需跨 UCIE 重新加载全部键。

5 验证方法与实验结果

5.1 独立参考模型与预训练权重

官方源码和第 20 层的五个索引器张量均固定到指定版本。通过 HTTP 范围请求，仅获取 5,707,008 字节张量数据，无需下载完整检查点。这些张量提供学习得到的 Q/K 投影、归一化参数与头权重。中间激活采用独立随机种子生成的高斯数据，并明确标记为合成输入；它们并非由文本驱动的整体模型推理中提取的隐藏状态。

独立参考模型将有限中间数表示为整数与 2 的整数次幂的乘积，并独立实现 IEEE FP32 舍入。对于加法和

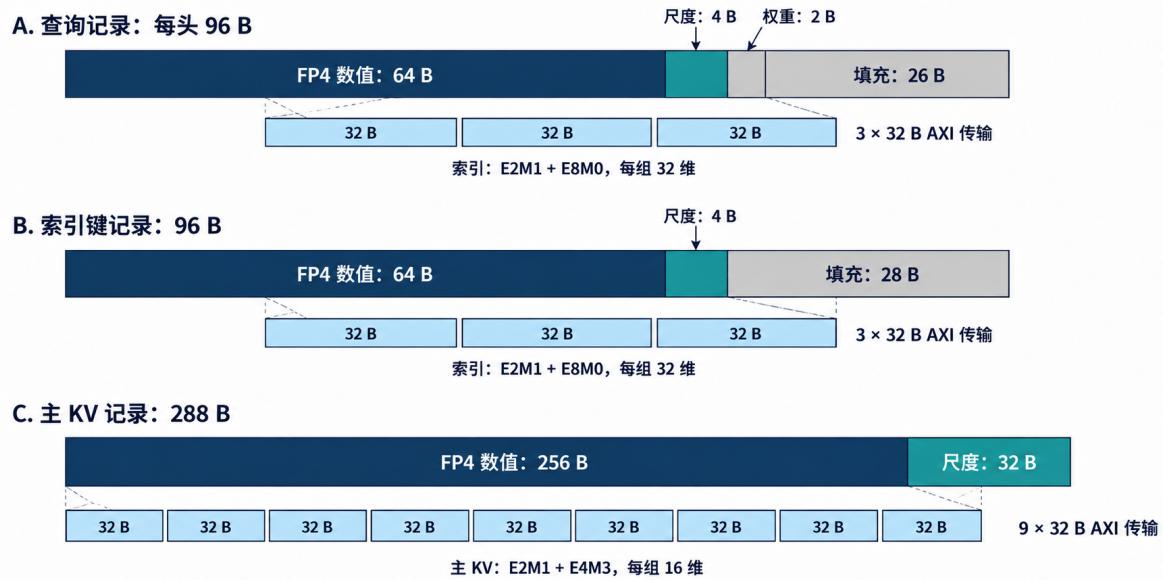


图 3: 物理记录格式示意，未按比例绘制。索引记录每 32 维采用一个 E8M0 缩放因子；主 KV 记录采用 E2M1 数值载荷，每 16 维配一个 E4M3 缩放因子。验收使用不解释数值含义的确定性 288 B 字节图案，验证选中 KV 的聚集读取与搬运，并非验证预训练模型主 KV 激活或注意力算术。填充字节和所有 AXI 数据拍均纳入统计。

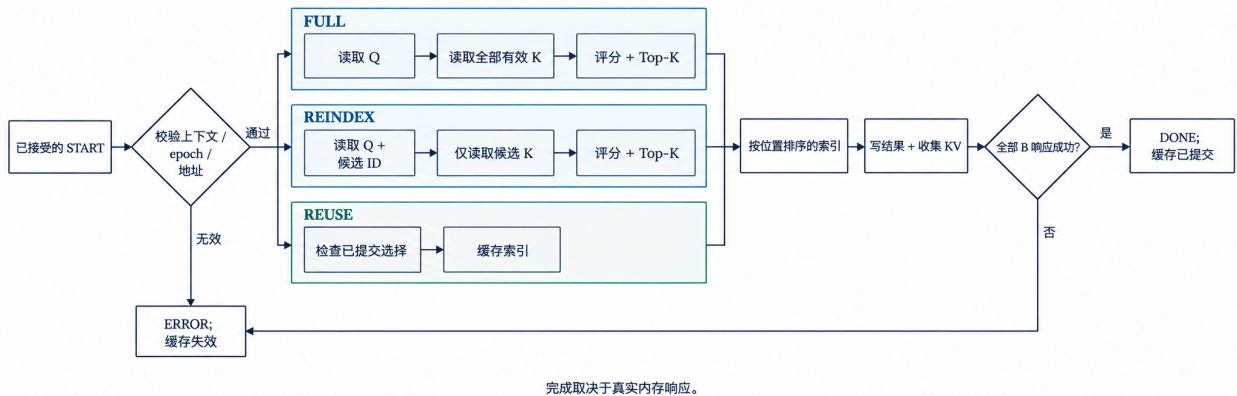


图 4: 已接受 START 的命令流程。候选块在硬件外展开；REUSE 跳过评分。DONE 等待全部写响应，命令失败使缓存失效；非法门铃写入在 START 前被拒绝。

乘法，分别使用 99,244 对有限输入与 NumPy 交叉核对；此外，还执行 5,010 个基于精确有理数的舍入区间检查。RTL 浮点运算模块通过 130,009 个用例验证，覆盖溢出与次正规数。通过验收的核心测试集包含 42 个用例、4,520 个成功键评分，以及实际执行的 13,557,824 个 FP4 乘积项。错误用例必须返回预期错误，且不产生选中结果。测试覆盖完整 H32/N640/K512 的预训练权重派生输入，以及仅扫描候选位置的输入。

对于采用预训练权重的 N640 核心用例，宿主参考计算以 FP32 执行官方的两个评分表达式，所得分数在该输入上与本文规定的顺序运算结果相同。BF16 表达式的相对 L2 差异为 0.00395188，最大绝对差异为 0.0328097，但 Top-512 集合一致。这只是一次独立算子观察，不能推广为普遍等价性或精度保证。在全零 N520 用例中，稳定选择较小索引的结果，与 Torch 未规定同分顺序的选择结果，其对称差集合包含 16 个元素，而这些元素的分数相同。

5.2 AXI 行为与在线验证

封装模块测试覆盖 W 先于 AW、AW 先于 W、随机 DMA 请求接受与响应延迟、从接口响应受阻、过期版本号、重复或越界候选、非法格式、读/写响应失败、输入输出缓冲区重叠，以及地址回绕。测试逐一核对选中分数和搬运后的每个字节，并检查通道停顿期间载荷是否稳定。该测试使用独立的按字节寻址测试存储器，与后续原生系统验证区分记录。

Vortex 命令处理器 DMA 在 H4/N16/K4 FULL 冒烟用例中发出 31 笔门控寄存器请求，宿主发出 0 笔。RTL 在 5201 周期后完成，DMA 读/写分别为 96 / 40 拍。被测程序核对 4 条分数/索引记录及全部 1152 B 聚集数据；更新排序逻辑后的在线回归仍得到相同结果。

同一在线链路还运行了固定的 H32/N640/K512 FULL 输入。索引头权重来自预训练第 20 层，激活仍为固定种子的合成数据；KV 聚集源是确定性字节图案。

表 1: 部分核心验证用例。C 表示仅扫描 N 个位置中的部分位置时提交的候选数。周期数包含规定的逐字节装载接口和测试背压, 不能视为系统延迟。

用例	H	N	K	周期数	检查
随机完整配置	32	521	512	362,348	通过
同分索引乱序	4	35	9	5,669	通过
有符号头权重	2	20	7	2,490	通过
次正规数得分	1	13	8	1,372	通过
候选子集 (C=22)	4	64	12	3,758	通过
权重: Top-512	32	640	512	444,514	通过
权重: 小例全扫描	32	64	8	46,101	通过
权重: C=16	32	64	8	13,250	通过

表 2: 同一 H32/N640/K512 输入和系统配置下的位置排序消融。周期由 RTL 波形状态与完成寄存器交叉核对; 不包含宿主启动、输入上传与结果回读。排序阶段包含收集 512 条结果的周期。

指标	插入排序基线	原位堆排序
排序相关周期	60,872	5,374
完整命令周期	670,720	615,173
DMA 读/写拍	6624 / 5120	6624 / 5120
已核对结果条数	512	512
已核对聚集字节	147,456	147,456

两个实现均由 Vortex CP DMA 发出命令, 宿主门控寄存器请求为 0; 被测程序逐项核对全部 512 条索引和 FP32 分数位模式, 以及 147,456 B 聚集输出。独立的 UCle / AXI、AoU 和 RTL 波形审计均通过, DONE 晚于最后一笔 DMA 写响应。每版命令均评分 640 个键、读取 6624 拍、写入 5120 拍, 没有因排序优化改变评分集合或存储流量。

排序相关阶段减少 55,498 周期, 完整命令减少 55,547 周期 (8.28%); 其余 49 周期差异分布在存储与输出等待状态中。两个版本的 KEYWAIT 均为 392,414 周期, 说明排序改动没有减少评分工作。改进版的排序阶段由 512 周期结果收集、4,350 周期堆调整和 512 周期最大位置取出构成。对于该命令, 选中 KV 的本地读与聚集写各为 4608 拍, 共占有 DMA 数据拍的 78.47%, 但这些拍数不是 UCle 上的远端传输量。命令忙区间内, UCle 请求没有直接访问 Q/K/KV 数据区; 本地门控 DMA 在链路的存储侧访问同一个在线内存后端。验收程序在命令开始前上传输入, 因而这项观察本身不构成相对计算侧放置的链路节省量或端到端加速证据。

该实验只有一组固定输入和一个系统配置, 且综合数据尚未覆盖新的排序封装。测得的 8.28% 是该配置的在线命令周期改进, 不能外推为完整模型吞吐率、能效或物理 PPA 改进。Vortex GPU 核函数周期仍为 0; 核函数直接发起门控命令尚未验证。

5.3 并行度与实现代价

当并行路数 L 能整除 32 时, 当前顺序调度所需的算术周期为

$$C_{\text{arithmetic/key}} = H \left[4 \left(\frac{32}{L} + 2 \right) + 3 \right]. \quad (8)$$

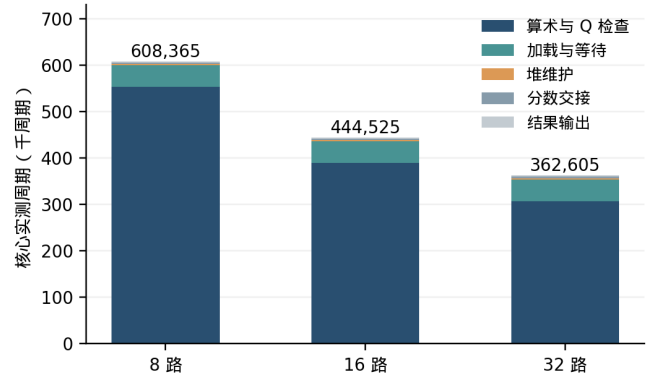


图 5: 同一预训练权重派生 H32/N640/K512 输入下, 运算通道数对核心周期数的影响。全部得分和选择结果一致。该核心独立测试不包含封装层的位置排序及原生 DMA / 存储等待; 结果输出周期取决于测试平台的 ready 序列。不同实现之间不预设时钟频率或 PPA 等价。

表 3: 16 路评分/堆核心在 TSMC 28 nm HPC+ 标准单元库下使用 Design Compiler 获得的实际综合映射结果。数组由触发器实现; 未完成布局布线、寄生互连提取、门级等价验证或物理签核。

指标	测量值/配置值
库工艺角	TT 0.9 V 25 °C
目标周期 (ns)	2.000
映射单元面积 (μm^2)	301,340.93
叶单元数	250,241
时序单元数	53,993
建立时间裕量 (ns)	+0.000006
保持时间裕量 (ns)	+0.005131
电气约束违例条目数	0
约束覆盖检查	通过

每组额外的两个周期分别用于数值转换和 FP32 加法; 每个头的其余开销包括查询有效性检查、权重乘法和跨头累加。当 H 为 32 时, L 为 8、16、32 分别对应 864、608、480 个周期。实测总延迟还包含字节加载、堆操作与接口停顿。以下图表采用实际统计周期, 而非以理想乘累加速率推算的延迟。在相同输入及相同握手调度下, 8 / 16 / 32 路实现分别需要 608,365、444,525 和 362,605 个周期。各实现均计算 2,621,440 个乘积项, 并选择相同的 512 个索引。从 8 路增加至 16 路、从 16 路增加至 32 路时, 优化前后周期数之比分别为 1.369 $\times$ 和 1.226 $\times$ 。增加运算通道可缩短整数乘积的分批计算时间, 但缩放转换与 FP32 归约仍串行执行。装载、选择和结果输出的开销也未消除, 因此该结果不意味着性能线性扩展, 亦不意味着不同实现具有相同的可达频率。

表 1 中 Top-512 用例的 444,514 个周期来自另一组随机背压序列, 与受控通道数实验中 16 路实现的 444,525 个周期不属于完全相同的时序实验。

在上述映射核心模型中, 报告的建立时间与保持时间检查均满足所给约束。但建立时间裕量仅为 6 fs, 保持时间裕量为 5.131 ps; 接近零的名义建立时间裕量不足以证明物理实现具有稳健的工作频率。电气约束违例条目统计转换时间、电容、扇出及其他报告的电气限制,

同一对象若出现在不同类别中则分别计数。该检查既不同于建立/保持时间检查,也不同于版图 DRC。独立的时钟与 I/O 约束覆盖检查通过,其中复位信号按文档规定设为假路径。组合单元面积为 $163,838.47 \mu\text{m}^2$,非组合单元面积为 $137,502.45 \mu\text{m}^2$,占总面积的 45.6%。后者同时包含控制状态与触发器存储,不能单独归因于 Q 或堆存储。全部 1,142 个 Liberty 单元的面积均与同名 LEF 单元的宽高乘积精确一致,工艺库发布版本的兼容关系亦已核对;实际 DC 数据库中的单元面积也逐项精确匹配。LEF SIZE 的尺寸单位为微米^[14],因此这里得到的是标准单元占用面积,而非布局后的核心面积或芯片面积。综合对象不包含 AXI 封装层、DMA、处理器模型及 UCIe 系统。综合使用理想时钟,未建模 CTS 与实际时钟网络负载;对于报告含 53,993 个负载的时钟网,工具采用 1,000 的扇出估计值。独立的已保存设计回读核查了映射与时序报告,但这不构成等价性证明。仿真时钟仅为测试设置,不代表已经实现的芯片工作频率。

5.4 瓶颈归因与放置条件

在受控 H32/N640/K512 输入下,16 路实现的算术阶段占 $389120/444525 = 87.5\%$;装载与等待阶段为 46,401 周期,堆调整阶段为 2,671 周期。将通道数从 16 加至 32,算术阶段从 389,120 降至 307,200 周期,总周期只获得 1.226 倍的前后比。以上是核心接口周期,并未包含在线 AXI / UCIe 往返、封装层位置排序或完整模型执行。改进后的在线命令则有 392,414 周期 (63.79%) 位于等待核心完成当前键的状态,61,440 周期 (9.99%) 用于逐字节装载键,108,508 周期 (17.64%) 位于 KV 聚集的请求/等待状态,排序相关阶段仅占 0.87%。这些互斥的封装状态统计说明进一步优化应瞄准评分、输入装载和聚集流水,而非继续单独优化位置排序。

设 C 为实际评分的键数, R 为选中数,链路每拍有效载荷为 32 B。当索引键必须从存储侧送往计算侧时,存储侧评分至多可以避免 3C 拍压缩键传输;查询上传、结果与选中 KV 流量以及远端算术时间仍需计入。当计算侧已驻留同一版本的索引键时,这项链路节省为零。故仅凭本系统的读写拍数,不能推出相对计算侧实现的加速;必须在相同数值合同、相同缓存驻留假设、相同候选集合和相同链路条件下测量两种放置策略。

6 讨论与局限

现有证据验证了可复现的算子算术,以及小规模冒烟输入和 H32/N640/K512 固定输入各一条 Vortex CP DMA 到在线存储的完整门控命令;尚不能确定完整模型的吞吐率、质量、能耗或量产芯片工作频率。Vortex 用例没有执行 GPU 核函数,因而不支持核函数直接发起命令或与门控并发争用的结论。

当前字节加载接口和最多仅允许一个未完成请求的 DMA 使部分工作串行执行。原位排序消除了基线的大部分位置排序周期,但在线命令中的 KEYWAIT 与聚集读写仍分别占大量周期;映射为触发器的状态存储也带来显著面积开销。我们另外用独立软件评估一种保守上界:以各组 Q 整数幅值和、K 最大幅值及缩放因子,为未执行的有符号加权 ReLU 头构造向上取整的 FP32 界;堆已满且候选上界必输时,才可提前停止。对现有真实权重加随机激活的 N640/K512 输入,该探索性模型只

为 14 个候选给出拒绝证书,避免 26 次头评分,占全部 20,480 次的 0.13%。将 K 改为 64 时,该比例为 4.11%;这不是官方配置或模型加速结果。模型与数据保存在仓库的 `research/assess_certified_pruning.py` 和 `evidence/research/certified_pruning_feasibility.json`。证书尚未进入 RTL,额外硬件成本未计入。由于简单上界收益有限,本稿不把提前终止列为已实现的性能贡献。

要将本工作发展为具有竞争力的体系结构论文,需要在真实长上下文解码轨迹上寻找并完成能显著降低评分成本的机制,再测量相同工作负载下计算侧驻留索引、计算侧远端读取索引和存储侧门控的端到端延迟、链路字节及完整面积/功耗。简单证书在当前 N640/K512 输入上的失败表明,不能只因为存在安全剪枝条件就推定它有系统收益。现有实验只能回答本文开头的可行性问题,不能替代这些对照。

7 研究材料与复现

源码、固定外部源码版本、RTL 测试、公开验证摘要及论文生成代码均在独立仓库中提供^[15]。仓库不分发模型权重或专有工艺库文件,而是提供限定范围的数据下载脚本和本地综合入口。概念图使用图片生成工具制作,并对照实现进行核查;通道数实验曲线与表格来源于保存的核心验证证据。本文为工程研究草稿,尚未经同行评审。

参考文献

- [1] DEEPSEEK-AI. DeepSeek-V4.1-Flash: Pushing the limits of KV cache compression[A/OL]. 2026. arXiv: 2609.19969. <https://arxiv.org/abs/2609.19969>.
- [2] DEEPSEEK-AI. DeepSeek-V4.1-Flash: Official model, configuration and inference reference[EB/OL]. 2026. <https://huggingface.co/deepseek-ai/DeepSeek-V4.1-Flash/tree/dba1be0a40aa45a94ad051997016db3960a90277>.
- [3] DEEPSEEK-AI. DeepSeek-V3.2: Pushing the frontier of open large language models[A/OL]. 2025. arXiv: 2512.02556. <https://arxiv.org/abs/2512.02556>.
- [4] StorageStacked contributors. StorageStacked: Unified online heterogeneous memory system[EB/OL]. 2026. <https://github.com/fmq03/StorageStacked/tree/3be39b697bdc315ae0be08ed2162c322bcf59462>.
- [5] WANG H, ZHANG Z, HAN S. SpAtten: Efficient sparse attention architecture with cascade token and head pruning[C/OL]//IEEE International Symposium on High-Performance Computer Architecture. 2021. <https://ieeexplore.ieee.org/document/9407232/>.
- [6] WANG H, WANG H, WANG Z, et al. PADE: A predictor-free sparse attention accelerator via unified execution and stage fusion[A/OL]. 2025. arXiv: 2512.14322. <https://arxiv.org/abs/2512.14322>.
- [7] JABER J, JABER O. StreamIndex: Memory-bounded compressed sparse attention via streaming top-

- k[A/OL]. 2026. arXiv: 2605.02568. <https://arxiv.org/abs/2605.02568>.
- [8] ZHOU R, MENG F, XU Y, et al. MISA: Mixture of indexer sparse attention for long-context llm inference [A/OL]. 2026. arXiv: 2605.07363. <https://arxiv.org/abs/2605.07363>.
- [9] CHENG L, ZHAO R, LIU T, et al. Guess-verify-refine: Data-aware top-k for sparse-attention decoding on blackwell via temporal correlation[A/OL]. 2026. arXiv: 2604.22312. <https://arxiv.org/abs/2604.22312>.
- [10] LIU S, XUE Y, TANG T, et al. Sample-guided exact top-k selection for long-context sparse attention[A/OL]. 2026. arXiv: 2609.08450. <https://arxiv.org/abs/2609.08450>.
- [11] XIE Z, HUANG Z, HUANG T, et al. HiSparse: Scaling sparse-attention decoding with hierarchical KV cache management[A/OL]. 2026. arXiv: 2608.07009. <https://arxiv.org/abs/2608.07009>.
- [12] BAI Y, DONG Q, JIANG T, et al. IndexCache: Accelerating sparse attention via cross-layer index reuse [A/OL]. 2026. arXiv: 2603.12201. <https://arxiv.org/abs/2603.12201>.
- [13] Open Compute Project. OCP Microscaling Formats (MX) Specification[EB/OL]. 2023. <https://www.opencompute.org/documents/ocp-microscaling-formats-mx-v1-0-spec-final-pdf>.
- [14] Cadence Design Systems. LEF/DEF 5.7 language reference[M/OL]. 2009. <https://www.ispd.cc/contests/18/lefdefref.pdf>.
- [15] StorageStacked-SparseGate contributors. StorageStacked-SparseGate: Reproducible RTL and online-system artifact[EB/OL]. 2026. <https://github.com/hy2581/StorageStacked-SparseGate>.